

## **Experiment: 01**

**Aim: Prepare a case study on Message Queuing Telemetry Transport (MQTT) Protocol.**

### **Introduction:**

The rapid expansion of the Internet of Things (IoT) has necessitated communication protocols that are far more efficient than traditional web standards like HTTP. With billions of devices—ranging from tiny sensors to massive industrial machines—connected globally, the network infrastructure often faces challenges such as low bandwidth, high latency, and unstable internet connections. To address these constraints, the Message Queuing Telemetry Transport (MQTT) protocol was developed. It is a standard designed specifically for machine-to-machine (M2M) communication in environments where network bandwidth is expensive or unreliable, and device power is limited.

### **What is Message Queue Telemetry Transport Protocol (MQTT)?**

MQTT is an extremely lightweight messaging protocol that operates on top of the TCP/IP stack. Unlike traditional protocols that send heavy headers and require constant "handshaking," MQTT is designed to be minimal. It is particularly effective for "constrained" devices—those with limited processing power, memory, or battery life. Because of its efficiency, it has become the de facto standard for connecting IoT devices in sectors like automotive telematics, manufacturing, and home automation.

### **History of MQTT**

MQTT was developed in 1999 by IBM engineers Andy Stanford-Clark and Arlen Nipper to monitor oil pipelines remotely in locations with limited network infrastructure. The initial version focused on providing lightweight and reliable messaging for telemetry systems. Over the years, MQTT gained traction in IoT applications due to its efficiency and simplicity. In 2013, MQTT 3.1 was standardized by the OASIS consortium, making it an official protocol for IoT communication. MQTT 5.0, released in 2016, introduced several enhancements including message expiry, reason codes, shared subscriptions, session expiry, and improved error reporting, further solidifying MQTT's role as a modern standard for IoT messaging.

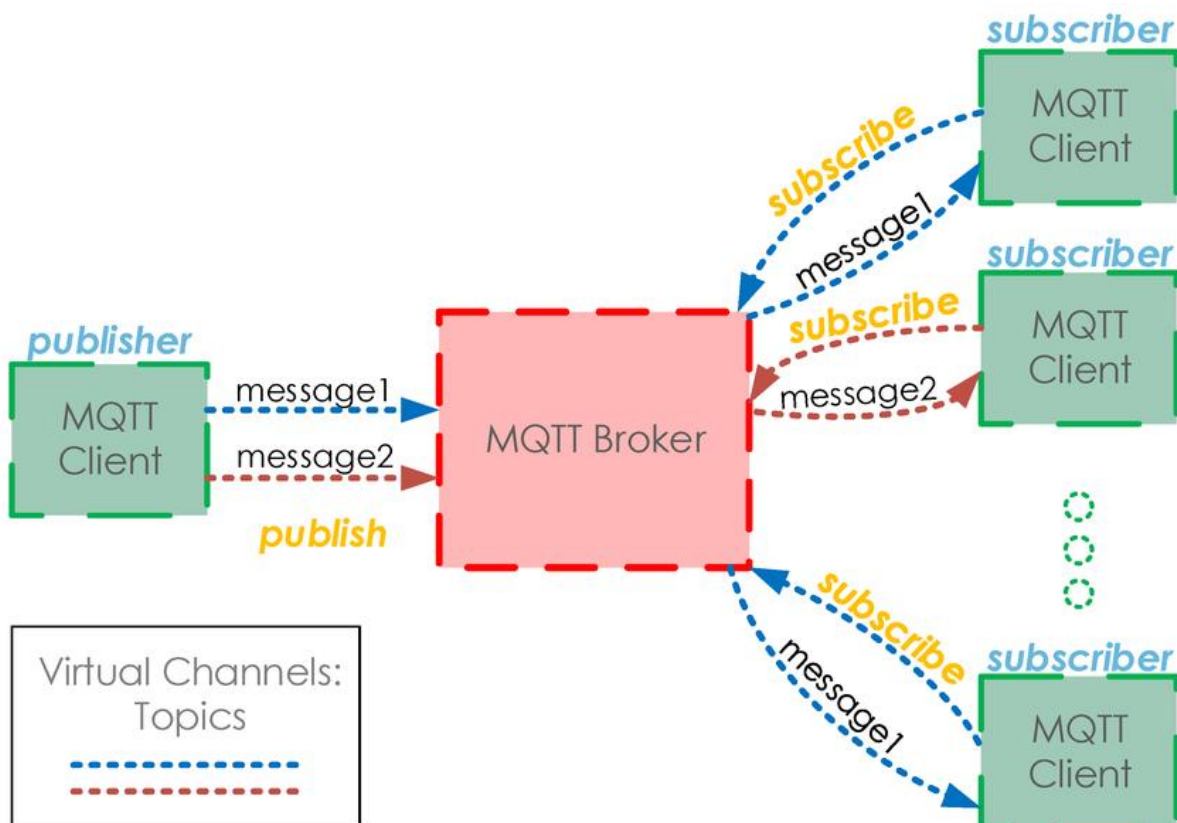
## Architecture and Workflow

MQTT follows a client-broker architecture, where communication occurs via a central server called the broker. Clients can act as publishers, sending messages to specific topics, or subscribers, receiving messages from topics they are interested in. The broker manages all communication, ensuring messages from publishers are delivered to the appropriate subscribers.

### The typical workflow involves:

1. **Client Connection:** The client establishes a connection with the broker using a CONNECT packet, optionally providing authentication credentials.
2. **Subscription:** Clients subscribe to topics of interest using a SUBSCRIBE packet.
3. **Publishing:** Publishers send messages to topics via the PUBLISH packet.
4. **Message Delivery:** The broker forwards the messages to all subscribers of the topic, according to the QoS level.
5. **Disconnection:** Clients can disconnect gracefully using the DISCONNECT packet, while the broker handles unexpected disconnections using the Last Will and Testament feature.

This architecture allows MQTT to support asynchronous communication, ensuring that publishers and subscribers do not need to be online simultaneously.



## **Working of MQTT**

MQTT operates on a client-broker architecture, where the broker acts as the central hub for all message exchange. Clients, which can be sensors, devices, or applications, communicate with each other by publishing messages to topics or subscribing to topics of interest. When a client wants to send data, it publishes a message to a specific topic on the broker. The broker then checks which clients are subscribed to that topic and delivers the message accordingly. This publish-subscribe model decouples the sender from the receiver, allowing for flexible and asynchronous communication.

The process begins when a client establishes a connection with the broker using a CONNECT message, optionally providing credentials for authentication. Once connected, clients can subscribe to topics by sending a SUBSCRIBE message. Whenever another client publishes a message to that topic, the broker forwards it to all subscribers. To ensure message reliability, MQTT provides three Quality of Service (QoS) levels: QoS 0 delivers messages at most once, QoS 1 ensures at least one delivery, and QoS 2 guarantees exactly one delivery using a handshake process.

MQTT also includes mechanisms like retained messages and Last Will and Testament (LWT) to improve reliability and monitoring. Retained messages allow the broker to store the last message of a topic so that new subscribers receive it immediately. LWT enables a client to specify a message that the broker will send on its behalf if the client disconnects unexpectedly, which is useful for alerting systems or monitoring devices. The broker manages all message routing and ensures that messages are delivered according to their QoS settings, providing efficient, reliable, and scalable communication even over unreliable networks.

Overall, the working of MQTT ensures real-time, lightweight, and asynchronous message exchange between multiple devices and applications, making it ideal for IoT systems, smart home devices, connected vehicles, healthcare monitoring, and other real-time data applications.

## **MQTT Message Structure**

MQTT messages consist of fixed headers, variable headers, and payloads. Key message types include:

- **CONNECT:** Initiates connection between client and broker.

- **PUBLISH:** Sends data to a topic.
- **SUBSCRIBE / UNSUBSCRIBE:** Registers or removes subscriptions for topics.
- **PINGREQ / PINGRESP:** Keeps the connection alive.
- **DISCONNECT:** Ends the connection.

Each message is compact, minimizing bandwidth usage and allowing operation in constrained environments.

## Quality of Service (QoS)

MQTT provides three levels of Quality of Service to ensure reliable message delivery:

- **QoS 0 (At Most Once):** Message is delivered at most once, with no acknowledgment. Suitable for non-critical messages.
- **QoS 1 (At Least Once):** Guarantees message delivery at least once, with acknowledgment required. Duplicate messages may occur.
- **QoS 2 (Exactly Once):** Ensures the message is delivered exactly once using a four-step handshake. Used for critical operations requiring high reliability.

These QoS levels allow developers to balance reliability against bandwidth and latency requirements.

## Security in MQTT

Security is critical in IoT systems. MQTT supports:

- **Authentication:** Username/password verification for client connections.
- **Encryption:** TLS/SSL encryption ensures that messages are transmitted securely.
- **Authorization:** Access control lists (ACLs) define which clients can publish or subscribe to specific topics.

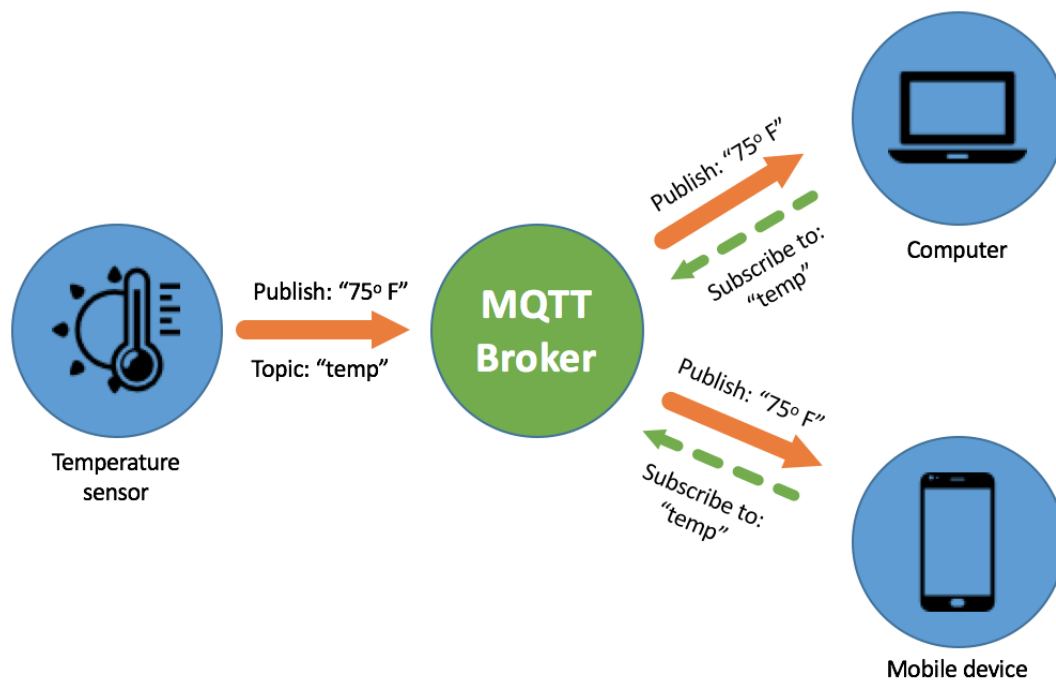
Despite these measures, MQTT security depends on proper broker configuration. Misconfigured brokers can expose data to unauthorized access or attacks.

## Real-Life Example: Smart Home Temperature Monitoring

In a modern smart home, various devices like thermostats, air conditioners, heaters, and temperature sensors need to communicate efficiently to maintain a comfortable environment. MQTT is an ideal protocol for this setup because it is lightweight, reliable, and works well over low-bandwidth networks, such as Wi-Fi or Zigbee-enabled devices.

**Scenario:**

A smart home system has multiple temperature sensors installed in different rooms, a central smart thermostat, and a mobile app for the homeowner. Each temperature sensor acts as an MQTT client (publisher), and the thermostat and mobile app act as subscribers to receive temperature updates in real time.

**Working Process:****1. Client-Broker Setup:**

All devices connect to a central MQTT broker hosted either locally or on a cloud service like HiveMQ or AWS IoT. Each device establishes a connection using the CONNECT message and authenticates if necessary.

**2. Publishing Data:**

each temperature sensor continuously measures the room temperature. Every few seconds, it sends a PUBLISH message to a topic specific to its location, such as home/livingroom/temperature or home/bedroom/temperature

**3. Subscribing to Topics:**

The smart thermostat subscribes to all temperature topics to adjust the heating or cooling system automatically. Similarly, the mobile app subscribes to receive updates, allowing the homeowner to monitor room temperatures in real time.

**4. Broker Message Handling:**

When a sensor publishes a temperature reading, the broker immediately forwards the message to all subscribed clients. If a subscriber is offline, the broker can use retained messages to store the last temperature reading so that the client receives it upon reconnecting.

**5. Quality of Service (QoS):**

QoS 1 or 2 can be used for important messages, ensuring that critical temperature updates are delivered at least once or exactly once. Less critical periodic updates can use QoS 0 to reduce network load.

**6. Last Will and Testament (LWT):**

If a sensor goes offline unexpectedly, the broker sends an LWT message to the thermostat and mobile app, alerting them that the sensor is unavailable. This allows the system to take corrective action, like triggering an alert or relying on nearby sensors.

**Characteristics of MQTT**

- **Lightweight:** MQTT is designed to be lightweight, making it suitable for use in aid-restrained environments inclusive of embedded systems and low-strength devices. The protocol minimizes bandwidth and processing overhead, enabling green communication even on restricted networks.
- **Publish-Subscribe Model:** In the publish-subscribe version, clients (publishers) send messages to subjects, and different clients (subscribers) acquire messages from subjects of interest. This decoupling of producers and purchasers permits for flexible and dynamic conversation styles.
- **Quality of Service (QoS) Levels:** MQTT supports exclusive stages of message delivery warranty, referred to as Quality of Service (QoS). QoS levels range from 0 to 2, providing various stages of reliability and message transport guarantees, relying at the utility necessities.
- **Retained Messages:** MQTT lets in agents to store retained messages on topics, making sure that new subscribers acquire the maximum latest message posted on a subject right now after subscribing. This characteristic is beneficial for fame updates and configuration settings.
- **Last Will and Testament (LWT):** MQTT clients can specify a Last Will and Testament message to be posted by way of the broker in the occasion of an sudden

consumer disconnect. This function affords a mechanism for detecting patron failures and dealing with them gracefully.

- **Security:** MQTT helps various protection mechanisms, consisting of Transport Layer Security (TLS) encryption and authentication mechanisms which include username/password and consumer certificates. These capabilities make certain the confidentiality, integrity, and authenticity of messages exchanged over MQTT connections.

### **Advantages of MQTT**

- Lightweight protocol that is quick to create and allows for efficient data transport
- Minimal data packet usage, resulting in low network usage
- Effective data dispersion
- The effective use of remote sensing and control
- Prompt and effective message delivery
- Minimizes power consumption, which is beneficial for the linked devices, and maximizes network capacity.

### **Disadvantages of MQTT**

- When compared to Constrained Application Protocol (CoAP), MQTT has slower send cycles.
- Resource discovery in MQTT is based on flexible topic subscription, while resource discovery in CoAP is based on a reliable system.
- MQTT lacks encryption. Rather, security encryption is accomplished by TLS/SSL (Transport Layer Security/Secure Sockets Layer).
- Building an internationally scalable MQTT network is challenging.
- Real-World Applications of MQTT
  - Smart homes (lights, fans, alarms)
  - Industrial IoT (predictive maintenance)
  - Healthcare (patient monitoring)
  - Vehicle Telematics & GPS
  - Smart agriculture
  - Oil & gas monitoring

## **Conclusion**

MQTT has become the backbone of modern IoT systems due to its lightweight nature, reliability, and ability to operate under low bandwidth conditions. Its publish–subscribe model allows highly scalable and flexible communication.

This case study shows that MQTT is essential for real-time IoT applications, especially where power, bandwidth, and reliability are major constraints.